Vision Career

Comprehensive Product, Technical, and Implementation Documentation

Professional master document for final project review, implementation, demo preparation, and future scaling

Project	Vision Career
Document purpose	Single-source reference for architecture, flows, AI logic, phases, data model, integrity rules, and implementation scope
Primary platform	Flutter mobile application
Core AI integrations	Gemini for recommendation / quiz / generation; Vertex AI Search for learning resources
Current product model	Phase-based guided path system: Phase 0 -> Phase 1 -> Phase 2 -> Phase 3

Table of Contents

1. Executive Summary 3

Why this system matters 3

2. Product Vision and Problem Statement 3

2.1 Product vision 3

2.2 Primary user problem 3

2.3 Product answer 4

2.4 Design philosophy 4

3. Core Experience and Student Journey 4

3.1 End-to-end student journey 4

3.2 Journey narrative 5

3.3 Product value for the student 5

4. Vision Career Phase Architecture 5

4.1 Phase overview 5

4.2 Phase 0 summary 5

4.3 Phase 1 summary 5

4.4 Phase 2 summary 6

4.5 Phase 3 summary 6

5. Detailed User Flows 6

5.1 Phase 0 flow architecture 6

5.2 Flow A - I Know What I Want 6

5.3 Flow B - I Don't Know Where I Fit 7

5.4 Tree interaction flow 7

5.5 FINAL PHASE flow 7

6. Functional Modules 7

7. AI Systems and External Services 8
7.1 High-level architecture....8
7.2 AI responsibilities....9
7.3 Control boundary....9
7.4 Resource ranking rules....9
7.5 Error handling expectations....10

8. Data Model and Persistence 10
8.1 Core data entities....10
8.2 Persistence rules....10
8.3 Recommended specialty registry....11

9. Quiz, Integrity, and Unlock Logic 11
9.1 Quiz role in progression....11
9.2 Assessment rules....11
9.3 Integrity controls....12
9.4 Unlock logic....12

10. Mobile Application Architecture 13
10.1 Architectural style....13
10.2 Major layers....13
10.3 Proposed runtime flow....13

11. File Structure and Responsibilities 13
11.1 High-level project tree....13
11.2 Phase 0 files....14
11.3 Final phase files....14

12. User Interface and Interaction Model 15
12.1 Tree design principles....15
12.2 Recommended state language....15
12.3 Node content expectations....15

13. Non-Functional Requirements 15
Reliability....15
Performance....15
Usability....15
Maintainability....15
Scalability....16

14. Risks, Constraints, and Mitigations 16

1. Executive Summary

Vision Career is a mobile-first guided education and career-planning system that helps university students move from uncertainty to job readiness through a structured, phase-based path. The product combines deterministic prerequisite logic with AI-assisted recommendation, assessment generation, and final-career preparation.

The system is intentionally designed so that large language models add intelligence without taking control. Models can rank, summarize, and generate suggestions, while the application remains responsible for validation, navigation, persistence, prerequisite enforcement, progress, and user safety.

At a product level, the experience is built around four major stages:

- Phase 0 - guidance and discovery before entering a path.
- Phase 1 - foundation subjects for the selected college or study area.
- Phase 2 - specialization subjects for the selected major.
- Phase 3 - final career-readiness topics generated from the user's completed path and target jobs.

Updated implementation reference: Implementation reference available at: https://github.com/khalilAmustafa/Vision-Career

This rebuild preserves the original documentation structure while integrating the evolved quiz system, AI decision layers, integrity monitoring signals, and final-phase path generation logic.

Why this system matters

- Students often choose majors and courses with weak guidance, incomplete market awareness, and no clear learning sequence.
- Traditional advising gives information, but not an interactive progression engine.
- Online learning platforms offer many resources, but no integrated academic path, assessment loop, or career-finish layer.
- Vision Career connects guidance, learning order, assessment, integrity, and final readiness in one coherent product.

---

2. Product Vision and Problem Statement

2.1 Product vision

Vision Career aims to become a smart academic navigation platform that does more than display majors. It should help a student discover where they fit, understand what to study next, prove they learned it, and complete a final bridge from specialization knowledge to career readiness.

2.2 Primary user problem

A typical student does not only need a list of subjects. They need a system that answers four deeper questions:

- Where do I fit?
- What should I study first, second, and later?
- How do I prove I understood each step?
- What do I still need before I am ready for work?

2.3 Product answer

Vision Career answers these questions with a combination of guided onboarding, prerequisite trees, node-level resources, quizzes with progression rules, and a final AI-assisted readiness stage.

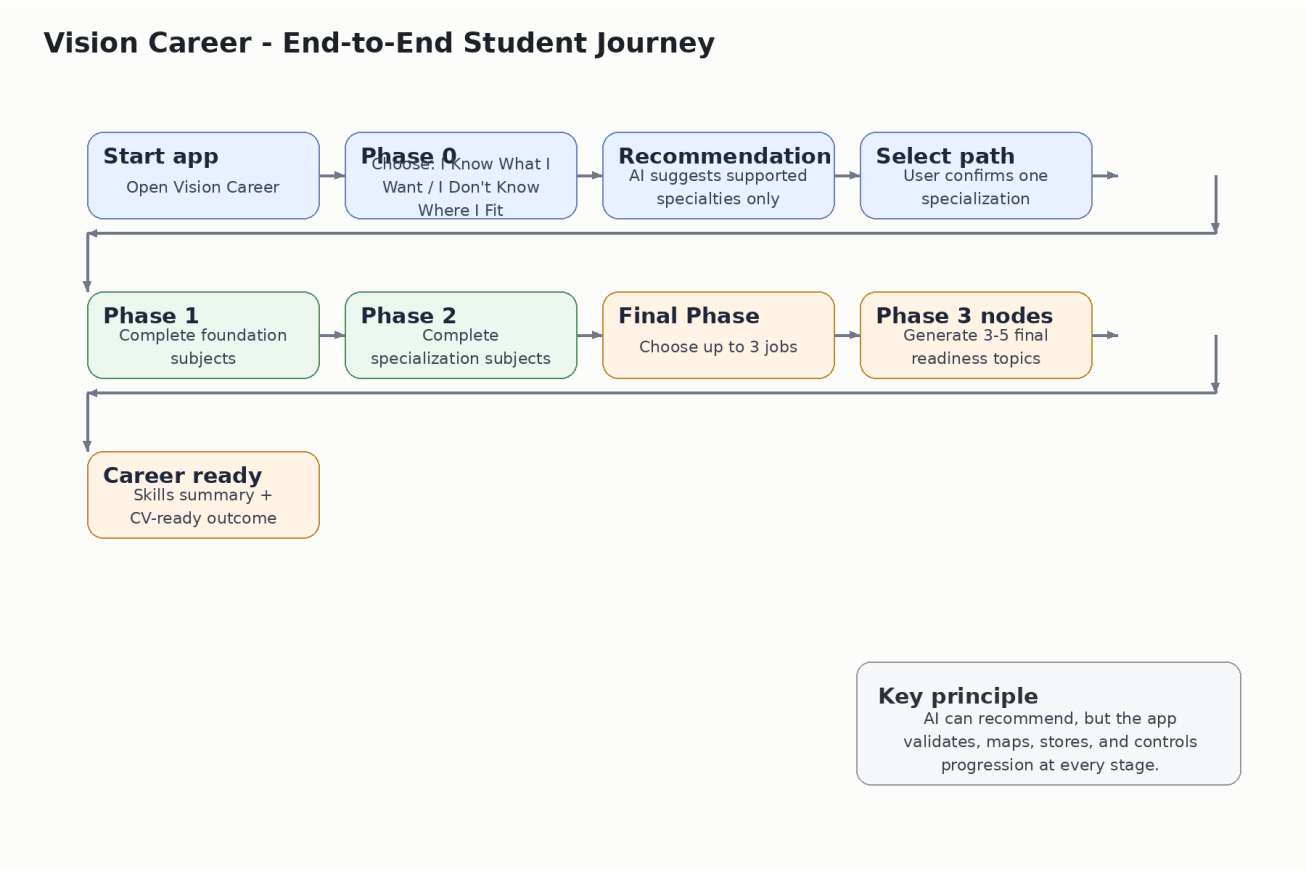
2.4 Design philosophy

- Impressive for demos, but technically safe in execution.
- AI-assisted, not AI-chaotic.
- User chooses; the app never auto-commits a path without explicit confirmation.
- Nodes are visual, interactive, and progression-based rather than passive text lists.
- Every major system must be recoverable, explainable, and testable.

---

3. Core Experience and Student Journey

3.1 End-to-end student journey



The student journey begins before path generation. Instead of forcing early manual selection, the system can guide the student into a suitable specialization and then keep them inside one persistent progression loop from discovery to job readiness.

3.2 Journey narrative

1. The student opens the app and enters Phase 0.
2. The student either describes what they already want or completes a guided discovery flow.
3. The app shows only supported specializations and requires the user to confirm one.
4. The tree opens with the correct local dataset and prerequisite graph.
5. The student explores node details, learning resources, gained skills, and requirements.
6. Completion is controlled by assessment rules; passing unlocks the next valid nodes.
7. Once Phase 1 and Phase 2 are complete, FINAL PHASE becomes available.
8. The student selects up to three possible jobs, receives a generated final mini-path, and completes the last readiness nodes.
9. The final output is a clearer job target, stronger skills summary, and a more CV-ready profile.

3.3 Product value for the student

Need	Typical student pain	Vision Career response
Direction	Unsure what major or specialty fits	Phase 0 recommendation flows
Order	Does not know subject sequence	Prerequisite-driven tree
Confidence	Cannot tell if they actually learned	Quiz gating and progression
Resources	Too many random learning links	Ranked resource retrieval
Career finish	Unsure what comes after specialization	Phase 3 career-readiness layer

4. Vision Career Phase Architecture

4.1 Phase overview

Phase	Purpose	Main input	Main output
Phase 0	Guidance and recommendation	User intent / discovery data	Selected supported specialty
Phase 1	Foundation learning	College / specialization mapping	Completed core subjects
Phase 2	Specialization progression	Phase 1 completion + prerequisites	Completed specialization subjects
Phase 3	Career readiness completion	Completed path + selected jobs	Final readiness nodes and skills summary

4.2 Phase 0 summary

Phase 0 replaces the older college-selection and specialization-selection entry route. It is the intelligent front door of the app. The goal is not to let the AI invent majors; the goal is to help the user choose from paths the app already supports.

4.3 Phase 1 summary

Phase 1 contains the foundational subjects that establish college-level or discipline-level readiness. These nodes should unlock in dependency-safe order and form the base of later specialization.

4.4 Phase 2 summary

Phase 2 contains specialization-specific subjects. These are the deeper technical or domain nodes that move the user toward a defined study area.

4.5 Phase 3 summary

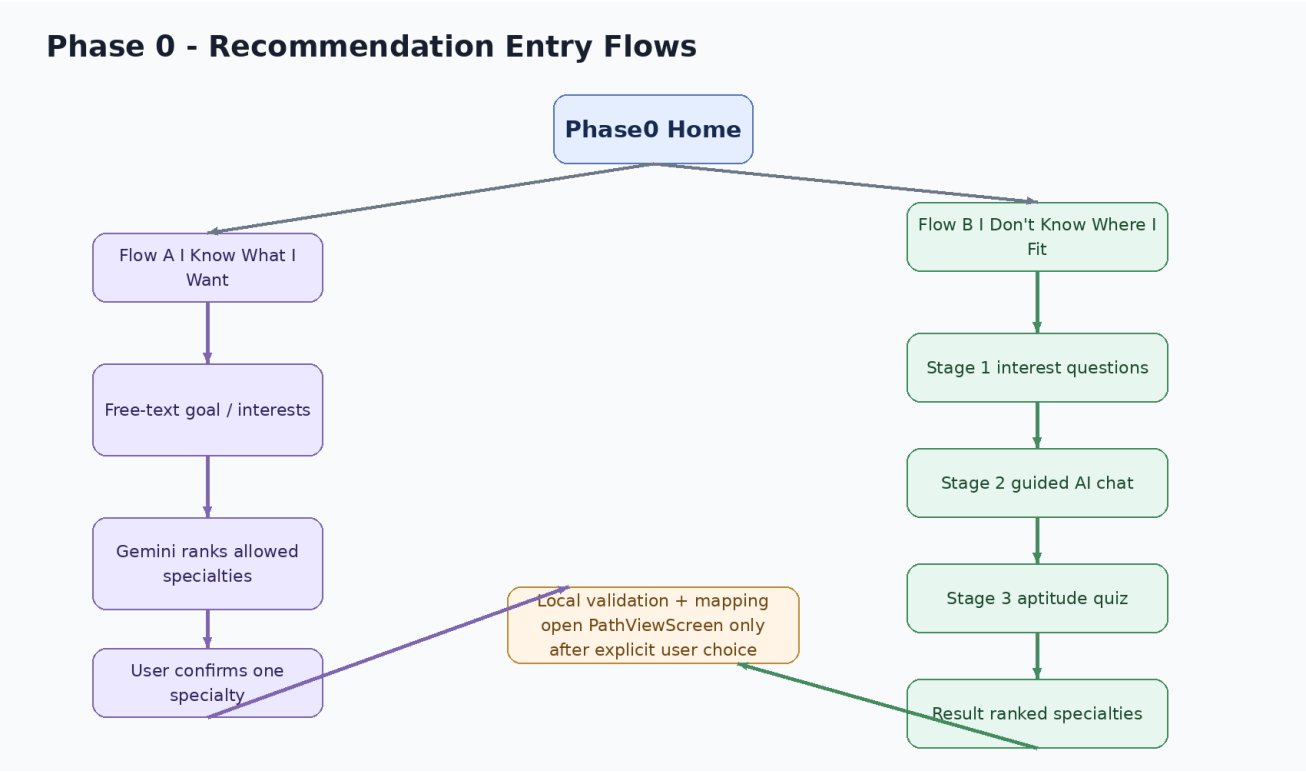
Phase 3 is the final transformation layer. After the learner completes the academic path, the system asks what job targets matter to them, then generates a small set of final topics that close practical readiness gaps.

Phase 3 now also connects career-role selection with generated final readiness nodes, ranked learning resources, and quiz-gated completion. Once generated and committed, the final mini-path is persisted as part of the user's product state and should not be regenerated casually.

---

5. Detailed User Flows

5.1 Phase 0 flow architecture



5.2 Flow A - I Know What I Want

1. User opens a free-text screen and describes what they want to study, build, or become.
2. The app sends the description together with the allowed specialty registry.
3. Gemini returns ranked specialties in strict JSON format.
4. The app filters invalid outputs, keeps only locally supported options, and displays them.
5. The user explicitly taps one recommendation to continue.
6. The app maps the specialty key to the local dataset and opens the tree.

5.3 Flow B - I Don't Know Where I Fit

7. Stage 1 collects lightweight interest questions using quick multiple-choice cards.
8. Stage 2 runs a bounded guided AI chat to clarify motivation, school preferences, and work style.
9. Stage 3 runs a soft aptitude quiz using simple math, logic, and reading signals.
10. The app sends summarized structured signals plus the allowed registry to Gemini.
11. Gemini returns ranked specialties from the allowed list only.
12. The app validates, displays, and waits for user confirmation before navigation.

5.4 Tree interaction flow

13. PathViewScreen displays the subject graph as an explorable tree.
14. Locked nodes remain visible and clickable so the student can inspect descriptions and requirements.
15. Unlocked nodes use a clear active visual state.
16. Completed nodes use a distinct completion state.
17. Long-press or controlled action opens the completion path, which is quiz-gated when required.
18. Dependency rules prevent invalid progression and should also prevent unsafe reverse actions that break consistency.

Updated completion gate: Node completion should follow the path Node -> Quiz Generation -> User Answer -> Evaluation -> Integrity Check -> Completion Decision -> Next Unlock. This keeps the tree deterministic while allowing each subject to receive a context-aware assessment.

5.5 FINAL PHASE flow

19. The FINAL PHASE button becomes available only after Phase 1 and Phase 2 are complete.
20. The app asks Gemini for a set of possible job roles using completed-path context.
21. The user can select up to three jobs.
22. The app asks Gemini for 3-5 final topics based on selected jobs, specialization, and completed subjects.
23. The app converts generated topics into new nodes that behave like normal learning nodes.
24. These nodes fetch resources, require quizzes, persist locally, and cannot be regenerated once committed.

---

6. Functional Modules

Module	Responsibility
Recommendation engine	Handles Phase 0 prompting, response parsing, local filtering, and safe retry behavior.
Mapping layer	Maps specialty keys to internal college / specialization / dataset identifiers.
Tree engine	Loads nodes, prerequisites, status, and unlock state for the chosen path.
Subject details layer	Displays descriptions, gained skills, status, resources, and completion actions.
Quiz engine	Builds or evaluates assessments, computes pass/fail, and returns result states.
Integrity layer	Applies anti-cheat controls and logs meaningful session flags.
Resource retrieval layer	Queries Vertex AI Search and ranks results using platform priority rules.

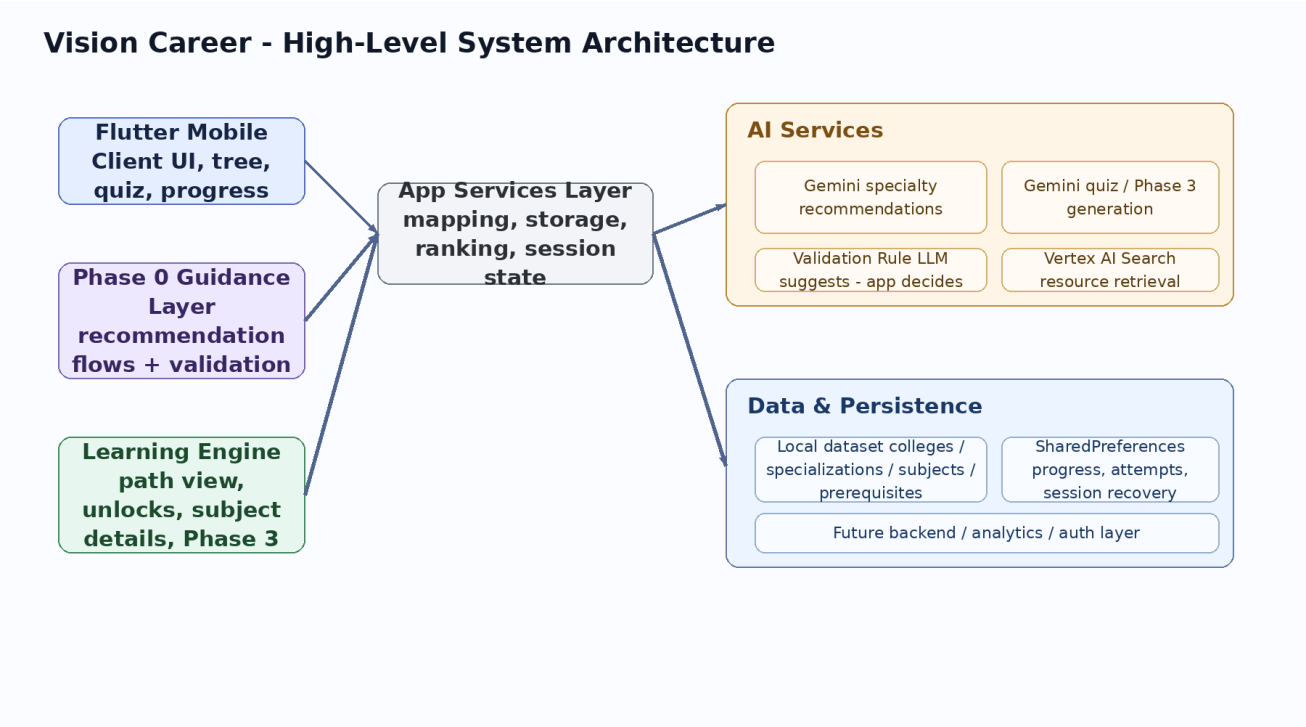
Phase 3 builder	Converts generated career-preparation outputs into persistent path nodes.
Storage layer	Persists progress, attempts, session recovery, and generated Phase 3 state.
Multi-type quiz engine	Generates and controls MCQ, text-based, and image-based quiz attempts per subject node.
Quiz evaluation service	Auto-checks MCQ answers, sends text answers for AI assessment, validates image submissions when enabled, and returns pass/fail decisions.
Exam session controller	Controls attempt limits, active quiz state, app-switch detection, screenshot protection, focus loss flags, and session recovery.
AI path generator layer	Uses graph-based / trained-model concepts to generate or support prerequisite-safe skill and readiness paths.

7. AI Systems and External Services

7.1 High-level architecture

Extended AI architecture keeps the original client-first model while formally separating candidate generation, validation, and product-state commitment.

```
Flutter Mobile Client
    ↓ service calls
Application Services (Mapping, Storage, Decision Logic, Unlock Engine)
    ↓ validated prompts / bounded contracts
Gemini Career AI | Gemini Quiz AI | Phase 3 Career Builder | Vertex AI Search
    ↓ candidate outputs only
Local Validation Layer (supported specialties, JSON schema, prerequisites, integrity rules)
    ↓ committed state
Local Progress Storage + Generated Phase 3 State
```



7.2 AI responsibilities

- Gemini in Phase 0: rank supported specialties and explain fit.
- Gemini in quizzes: generate or assess question sets according to the subject context.
- Gemini in Phase 3: suggest job roles and generate final readiness topics.
- Vertex AI Search: return ranked learning resources for node topics.

• Career AI: receives user intent or discovery signals and returns ranked specialties from the allowed registry only.

• Path Generator / GNN concept: supports prerequisite-safe path ordering, skill coverage, dependency validation, and future model-based path optimization.

• Quiz Generator AI: generates MCQ, text-answer, or image-answer tasks using the selected subject title, description, skills, phase, and prerequisites as context.

• Integrity AI / signals: combines phone detection, face presence checks, focus-loss events, app switching, and screenshot protection into a session-level integrity decision.

7.3 Control boundary

A strict boundary should remain in place across the whole system: LLMs produce candidate outputs; the application validates and decides what becomes real product state.

Decision boundary update: AI output must never directly unlock nodes, overwrite progress, create unsupported majors, or bypass prerequisite rules. The app must validate JSON shape, allowed values, quiz thresholds, integrity flags, and persistence rules before committing any state.

7.4 Resource ranking rules

- Maximum four resources per node.
- Maximum two Coursera results.

- Platform priority: Coursera -> Udemy -> YouTube.
- The app should deduplicate and discard weak or irrelevant results.
- Topic generation and resource retrieval remain separate responsibilities.

7.5 Error handling expectations

- Malformed JSON should trigger parse-safe fallback and retry UI.
- Invented majors or unknown specialty keys must be dropped.
- Empty valid results must not silently continue; the user should receive a clear retry option.
- External API failures should not corrupt saved progress or route state.

---

8. Data Model and Persistence

8.1 Core data entities

Entity	Fields	Purpose
College	id, name	Top-level academic grouping
Specialization	id, college_id, key, title	Selectable path identity
Subject / node	id, title, phase, credits, description, skills	Learning unit shown in the tree
Prerequisite edge	subject_id, prerequisite_subject_id	Dependency relation for unlocking
User progress	user_id, subject_id, completed, score, attempts	Progress and result tracking
Phase 0 session	answers, summaries, chosen specialty	Recovery for unfinished recommendation flows
Phase 3 state	selected_jobs, generated_nodes, completion states	One-time final-phase persistence
Quiz session	session_id, user_id, subject_id, quiz_type, started_at, status, attempt_no	Tracks an active or completed quiz attempt
Quiz item	question, type, choices, correct_index, rubric, difficulty_tag	Stores generated assessment content before evaluation
Quiz result	score, passed, feedback, evaluator, integrity_status	Stores assessment outcome and unlock eligibility
Integrity event	session_id, event_type, severity, timestamp, metadata	Stores meaningful anti-cheat flags only

8.2 Persistence rules

- Normal path progress should persist independently from temporary Phase 0 session data.
- Phase 0 should support mid-flow recovery after app close or interruption.
- Phase 3 generated nodes must be stored once committed and must not be regenerated casually.
- Attempt limits and integrity-related counters should be stored only as far as they are actually used in product logic.

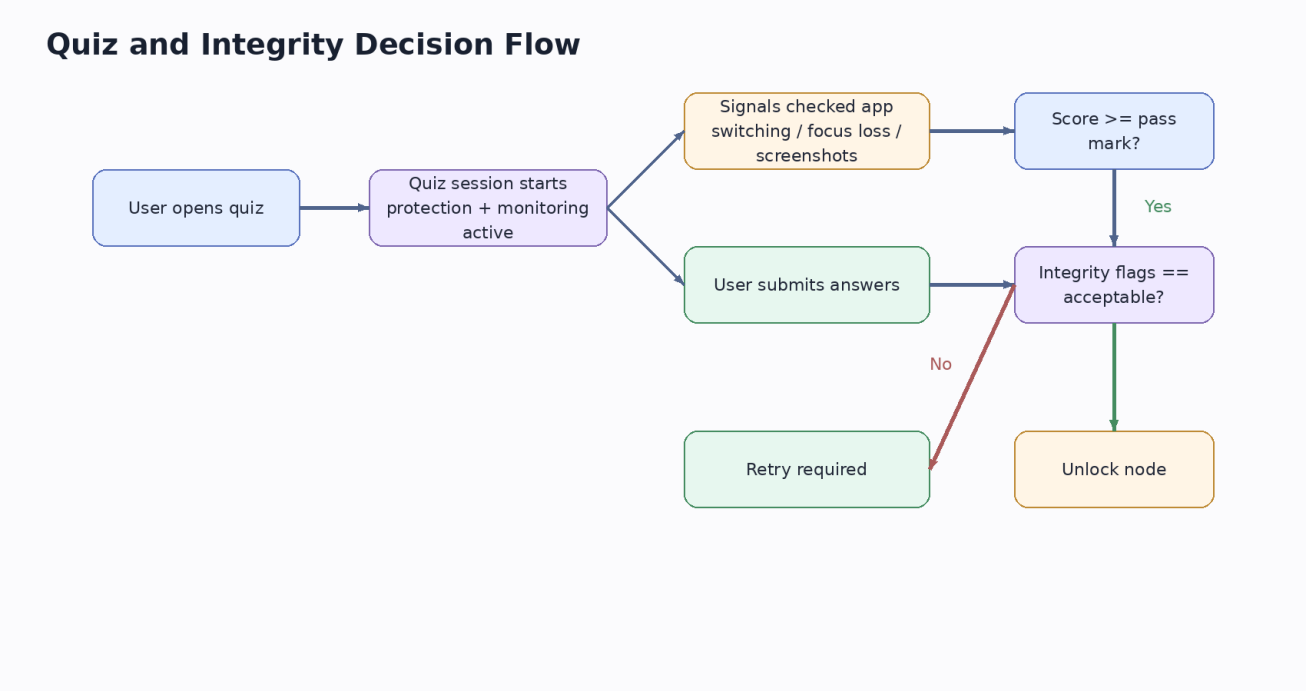
Quiz persistence should store the minimum required data for product decisions: attempt count, result status, pass/fail, feedback summary, and integrity decision. Raw camera data or noisy logs should not be stored unless a future feature explicitly uses them.

8.3 Recommended specialty registry

The application should maintain a single source of truth for all specialties Gemini is allowed to suggest. This registry protects the product from model drift and invented outputs.

---

9. Quiz, Integrity, and Unlock Logic



9.1 Quiz role in progression

Quizzes are not decorative. They are a formal progression gate. Their role is to verify that the student is ready to unlock the next nodes.

The updated quiz system supports three assessment types. MCQ is used for direct conceptual checks. Text-based answers are used for explanation, reasoning, short writing, or code-like responses. Image-based answers are used when a subject requires visual evidence, diagram interpretation, design output, handwritten work, or screenshot-style submission.

Each quiz is generated using subject context: node title, phase, description, skills, prerequisites, and current user progress. This prevents generic questions and keeps the assessment aligned with the exact learning node.

9.2 Assessment rules

- A node should only complete after the required quiz condition is satisfied.
- Passing threshold should be configurable by phase or quiz type.

- Integrity flags must be checked before accepting a passing result.
- Failure should trigger a clear retry path instead of ambiguous state.

MCQ evaluation: the app can auto-check selected answers against correct_answer_index and calculate the score immediately.

Text evaluation: the app sends the user's written answer, subject context, and rubric to Gemini. Gemini returns a structured score, feedback, weakness tags, and pass/fail recommendation. The app still applies the final decision rule.

Image evaluation: when enabled, the app accepts an image submission and validates it against the expected task using AI-assisted feedback or manual-compatible rubric fields. The product should store the result summary, not unnecessary raw evidence.

Adaptive difficulty: difficulty can be selected from subject phase, previous attempts, and past performance. If adaptation is not fully implemented, the JSON contract should still include difficulty_tag so the system can evolve safely.

9.3 Integrity controls

- App switching and focus loss detection during quiz sessions.
- Screenshot protection and anti-copy behavior during assessments.
- Session flagging for abnormal behavior.
- Minimal and useful logging only; avoid storing noise that has no product value.

Integrity decision logic should combine quiz score with integrity severity. A high score should not automatically unlock the next node when the session contains severe violations such as repeated app switching, focus loss, abnormal face absence, or phone detection during the assessment window.

9.4 Unlock logic

- Locked nodes remain visible but unavailable for completion.
- Unlocked nodes appear only when prerequisites are satisfied.
- Completed nodes should not be allowed to revert in a way that breaks downstream dependency consistency.
- The UI should clearly communicate why a node is locked and what is required next.

```
IF prerequisites_satisfied
   AND quiz_passed
   AND integrity_flags <= allowed_threshold
THEN mark current node completed and unlock eligible next nodes
ELSE keep node incomplete and show retry / requirement feedback
```

Updated quiz JSON contract should support multiple quiz types while keeping a stable parseable shape:

```
{
  "subject_id": "string",
  "quiz_type": "mcq | text | image",
  "difficulty_tag": "easy | medium | hard",
  "questions": [
    {
      "question": "string",
      "choices": ["A", "B", "C", "D"],
      "correct_answer_index": 0,
      "rubric": "string",
```

```
      "expected_image_evidence": "string",
      "explanation": "string"
    }
  ],
  "passing_score": 50
}
```

---

10. Mobile Application Architecture

10.1 Architectural style

The current product is a Flutter mobile application with local-first behavior and optional cloud-backed services. It mixes deterministic path logic with selective AI calls and should be treated as a modular client application rather than a thin UI shell.

10.2 Major layers

- Presentation: screens, widgets, navigation, interaction states.
- Application / services: mapping, prompting, storage, ranking, unlock evaluation.
- Data: local JSON or structured data files for colleges, specializations, subjects, and prerequisites.
- External integrations: Gemini and Vertex AI Search.

Repository alignment: the mobile implementation should keep Gemini configuration, Vertex AI Search configuration, career services, storage services, Phase 0 services, and Phase 3 builders separated under the existing Flutter structure. Implementation reference available at: https://github.com/khalilAmustafa/Vision-Career

10.3 Proposed runtime flow

25. UI action occurs.
26. Screen calls service or repository layer.
27. Service validates local state and external contract input.
28. External AI or search response returns.
29. Service parses and filters output.
30. UI updates only after a safe local product state is formed.

---

11. File Structure and Responsibilities

11.1 High-level project tree

```
vision_career_mobile/
|- lib/
|   |- app/
```

```
|   |- core/
|   |   |- constants/
|   |   |- services/
|   |   |- models/
|   |   |- widgets/
|   |- data/
|   |   |- datasources/
|   |   |- repositories/
|   |   |- models/
|   |- features/
|   |   |- phase0/
|   |   |- path_view/
|   |   |- subject_details/
|   |   |- progress/
|   |   |- career/        # final phase
|   |- shared/
|- assets/
   |- data/
   |- images/
   |- icons/
   |- json/
```

11.2 Phase 0 files

- phase0_home_screen.dart - two-option landing screen.
- know_what_i_want_screen.dart - free-text collection.
- specialty_recommendation_screen.dart - recommendation display and confirmation.
- fit_discovery_questions_screen.dart - stage 1 interest questions.
- fit_discovery_chat_screen.dart - stage 2 guided AI chat.
- fit_discovery_quiz_screen.dart - stage 3 aptitude quiz.
- fit_discovery_result_screen.dart - ranked results and final handoff.
- phase0_gemini_service.dart - prompt building, parse logic, validation.
- phase0_mapping_service.dart - specialty key to dataset mapping.
- phase0_session_storage_service.dart - recoverable temporary state.

11.3 Final phase files

- career_selection_screen.dart - job-role selection UI.
- career_analysis_screen.dart - generated topic result flow.
- career_llm_service.dart - Phase 3 prompting.
- career_phase3_builder.dart - convert generated topics into nodes.
- career_storage_service.dart - persistent Phase 3 state.

---

quiz_generation_service.dart - builds subject-aware quiz prompts and parses quiz JSON.

quiz_evaluation_service.dart - evaluates MCQ locally and delegates text / image assessment when needed.

quiz_attempt_storage_service.dart - persists attempt count, pass/fail, feedback, and integrity decision.

integrity_session_service.dart - tracks app switching, focus loss, screenshot protection, and supported vision-based integrity signals.

---

12. User Interface and Interaction Model

12.1 Tree design principles

- Tree should feel explorable and visual, not like a static list.
- Locked, unlocked, and completed states should have clearly different treatments.
- Wide trees should support panning or zoom-friendly interaction rather than forcing cramped layouts.
- Node details should remain accessible even when a node is locked.

12.2 Recommended state language

State	Meaning	Recommended visual
Locked	Prerequisites not yet satisfied	Muted gray / shadowed
Unlocked	Available to attempt	Active blue
Completed	Requirement satisfied	Gold / highlighted success state

12.3 Node content expectations

- Title
- Description
- Skills gained
- Prerequisites / unlock requirements
- Learning resources
- Completion action and status

---

13. Non-Functional Requirements

Reliability

- The app should preserve progress correctly after closes and restarts.
- AI failures should degrade gracefully without corrupting path state.

Performance

- Tree rendering should remain responsive on mobile devices.
- External calls should display clear loading states and retry actions.

Usability

- Flows must feel short, understandable, and visual.
- Error states must be user-readable, not developer-style messages.

Maintainability

- Service responsibilities should stay separated.
- AI contracts should be explicit and versionable.

Scalability

- The system should support more colleges and specializations without rewriting the entry architecture.
- The recommendation registry should make multi-college expansion safer.

---

14. Risks, Constraints, and Mitigations

Risk	Why it matters	Mitigation
LLM invents unsupported majors	Can break navigation and trust	Use allowed specialty registry and local filtering
Malformed AI output	Can crash or confuse flows	Strict JSON contracts plus retry state
Feature creep late in project	Can reduce stability before demo	Freeze architecture and focus on polish
Weak data quality	Hurts recommendation and path credibility	Clean priority specializations and validate datasets
Over-logging integrity data	Adds complexity without value	Store only signals actually used by product logic
Quiz hallucination or weak questions	Can make progression unfair or untrusted	Use strict quiz JSON contracts, subject context, rubrics, and retry / validation fallback
Integrity false positives	Can block honest students	Use severity thresholds, clear feedback, and avoid relying on one weak signal alone
Generated Phase 3 nodes drift from supported data	Can weaken final career path credibility	Persist generated nodes once, validate structure, and keep resource retrieval separate from generation